Estratégias **Empíricas**

Prof. Paulo Roberto Farah
Engenharia de Sotware

UDESC
UNIVERSIDADE
DO ESTADO DE
SANTA CATARINA

Paradigmas de Pesquisa

1) Pesquisa Qualitativa

- Estuda objetos no seu formato natural
- Começa aceitando que há diversas maneiras de interpretação
- Interpreta um fenômeno a partir das explicações das pessoas
- Preocupa-se em descobrir as causas percebidas pelos sujeitos do estudo e entender sua visão do problema em questão
- O sujeito é a pessoa que participa de um experimento para avaliar um objeto

Pesquisa Qualitativa

2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)

A Qualitative Study on the Implementation Design Decisions of Developers

Jenny T. Liang
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA, USA
jtliang@cs.cmu.edu

Maryam Arab
Department of Computer Science
George Mason University
Fairfax, VA, USA
marab@gmu.edu

Minhyuk Ko
Department of Computer Science
Virginia Tech
Blacksburg, VA, USA
minhyukko@vt.edu

Amy J. Ko
Information School
University of Washington
Seattle, WA, USA
ajko@uw.edu

Thomas D. LaToza
Department of Computer Science
George Mason University
Fairfax, VA, USA
tlatoza@gmu.edu

***Abstract*—Decision-making is a key software engineering skill. Developers constantly make choices throughout the software development process, from requirements to implementation. While prior work has studied developer decision-making, the choices made while choosing what solution to write in code remain understudied. In this mixed-methods study, we examine the phenomenon where developers select one specific way to implement a behavior in code, given many potential alternatives. We call these decisions *implementation design decisions*. Our mixed-methods study includes 46 survey responses and 14 semi-structured interviews with professional developers about their decision types, considerations, processes, and expertise for implementation design decisions. We find that implementation design decisions, rather than being a natural outcome from higher levels of design, require constant monitoring of higher level design choices, such as requirements and architecture. We also show that developers have a consistent general structure to their implementation decision-making process, but no single process is exactly the same. We discuss the implications of our findings on research, education, and practice, including insights on teaching developers how to make implementation design decisions.**

***Index Terms*—implementation design decisions, software design**

I. Introduction

Making decisions effectively is a crucial skill for software engineers [1]. One reason is because making explicit and rationalized design decisions during the design process improves software design quality [2]. Developers make these explicit decisions throughout the software design process, from requirements and architecture to implementation. These decisions in turn have downstream effects on the software, such as influencing how easily developers comprehend a codebase [3] or resulting in systems that are difficult to maintain [4], [5].

At higher levels of software design, developers make explicit decisions about the software architecture by prototyping them at the whiteboard [6] or documenting them in UML diagrams [7]. Researchers have developed an understanding of how developers make architectural decisions [8], [9], even building several tools to aid this process [10], [11]. At the code level, developers also make explicit decisions, such as implementing specific design patterns [12], [13]. Yet, the decisions that developers make while choosing what solution to write in code remains understudied. We call these decisions *implementation design decisions*.

Implementation design decisions are when developers select one specific way to implement a behavior, given many potential alternatives. They may choose an implementation that minimizes the time to market in place of producing high-quality, robust software. Meanwhile, developers may choose

Fig. 1. An example of an implementation design decision. Developers consider aspects of software design that are above the implementation (e.g., requirements, architecture, or class and modules) to make these decisions.

1558-1225/23/$31.00 ©2023 IEEE
DOI 10.1109/ICSE48619.2023.00047

Who Answers It Better? An In-Depth Analysis of ChatGPT and Stack Overflow Answers to Software Engineering Questions

Samia Kabir
Purdue University
West Lafayette, USA
kabirs@purdue.edu

David N. Udo-Imeh
Purdue University
West Lafayette, USA
dudoimeh@purdue.edu

Bonan Kou
Purdue University
West Lafayette, USA
koub@purdue.edu

Tianyi Zhang
Purdue University
West Lafayette, USA
tianyi@purdue.edu

ABSTRACT

Q&A platforms have been an integral part of the web-help-seeking behavior of programmers over the past decade. However, with the recent introduction of ChatGPT, the paradigm of web-help-seeking behavior is experiencing a shift. Despite the popularity of ChatGPT, no comprehensive study has been conducted to evaluate the characteristics or usability of ChatGPT's answers to software engineering questions. To bridge the gap, we conducted the first in-depth analysis of ChatGPT's answers to 517 Stack Overflow (SO) questions and examined the correctness, consistency, comprehensiveness, and conciseness of ChatGPT's answers. Furthermore, we conducted a large-scale linguistic analysis, and a user study to understand the characteristics of ChatGPT answers from linguistic and human aspects. Our analysis shows that 52% of ChatGPT answers are incorrect and 77% are verbose. Nonetheless, ChatGPT answers are still preferred 39.34% of the time due to their comprehensiveness and well-articulated language style. Our result implies the necessity of close examination and rectification of errors in ChatGPT, at the same time creating awareness among its users of the risks associated with seemingly correct ChatGPT answers.

CCS CONCEPTS

• **Software and its engineering**; • **General and reference** → **Empirical studies**;

KEYWORDS

stack overflow, q&a, large language model, chatgpt

ACM Reference Format:
Samia Kabir, David N. Udo-Imeh, Bonan Kou, and Tianyi Zhang. 2018. Who Answers It Better? An In-Depth Analysis of ChatGPT and Stack Overflow Answers to Software Engineering Questions . In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXXXXXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2018 Association for Computing Machinery.
ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Software developers often resort to online resources for a variety of software engineering tasks, e.g., API learning, bug fixing, comprehension of code or concepts, etc. [53, 57, 63]. A vast majority of these help-seeking activities include frequent engagement with community Q&A platforms such as Stack Overflow[1](SO) [52, 53, 62, 63] to seek help, solutions, or suggestions from other developers.

The emergence of *Large Language Models (LLMs)* has demonstrated the potential to transform the web help-seeking patterns of software developers. Recent studies show that programmers utilize AI tools such as GitHub Copilot [27] for faster exploration and queries of problems at hand and turn to web searches or SO only when they need to verify a solution or access the documentation [6, 55, 61]. The ability to engage in interactive conversations and provide apt solutions using natural language has propelled *LLMs* into becoming a popular option among programmers.

In continuation of *LLM* progress, in November 2022, ChatGPT [44] was introduced as an open-access Chatbot, which surpassed the popularity of other models in its category. ChatGPT's capacity to engage in human-like conversations, promptly learn from continuous human feedback, and accessibility to the general public, have all contributed to its popularity. Consequently, ChatGPT's popularity has ignited numerous debates among academics, researchers, and industry professionals on Twitter and other social media platforms [17, 51]. These discussions revolve around the scenarios wherein ChatGPT could potentially replace prominent search engines (e.g., Google) or widely used Q&A platforms (e.g., Stack Overflow).

Despite the increasing popularity of ChatGPT, concerns surrounding its nature as a generative model and the associated risks remain prevalent. Previous studies [8, 24, 26, 28, 41] show that *LLMs* can acquire and propagate factually incorrect knowledge, which can persist in their generated or summarized texts. Additionally, *LLMs* often generate fabricated texts that mimic truthful information, thereby introducing risk for non-expert end-users who lack the means to verify factual inconsistencies [11, 16, 21]. Recent studies show that ChatGPT is also plagued with these issues [12, 30, 36, 43]. The prevalence of misinformation, which can easily mislead users, has prompted Stack Overflow to impose a ban on posting answers generated by ChatGPT [46].

[1]https://stackoverflow.com/

UDESC

Paradigmas de Pesquisa

2) Pesquisa Quantitativa

- Preocupa-se em quantificar um relacionamento ou comparar dois ou mais grupos
- O objetivo é identificar uma relação de causa e efeito
- É conduzida através da criação de experimentos controlados ou coleta de dados por meio de estudos de caso
- São apropriadas ao testar o efeito de alguma manipulação ou atividade
- Uma vantagem é que os dados quantitativos promovem comparações e análises estatísticas

Pesquisa Quantitativa

Do Static Analysis Tools Affect Software Quality when Using Test-driven Development?

Simone Romano
University of Salerno
Italy
siromano@unisa.it

Fiorella Zampetti
University of Sannio
Italy
fiorella.zampetti@unisannio.it

Maria Teresa Baldassarre
University of Bari
Italy
mariateresa.baldassarre@uniba.it

Massimiliano Di Penta
University of Sannio
Italy
dipenta@unisannio.it

Giuseppe Scanniello
University of Salerno
Italy
gscanniello@unisa.it

ABSTRACT

Background. Test-Driven Development (TDD) is an agile software development practice, which encourages developers to write "quick-and-dirty" production code to make tests pass, and then apply refactoring to "clean" written code. However, previous studies have found that refactoring is not applied as often as the TDD process requires, potentially affecting software quality.

Aims. We investigated the benefits of leveraging a Static Analysis Tool (SAT)—plugged-in the Integrated Development Environment (IDE)—on software quality, when applying TDD.

Method. We conducted two controlled experiments, in which the participants—92, in total—performed an implementation task by applying TDD with or without a SAT highlighting the presence of code smells in their source code. We then analyzed the effect of the used SAT on software quality.

Results. We found that, overall, the use of a SAT helped the participants to significantly improve software quality, yet the participants perceived TDD more difficult to be performed.

Conclusions. The obtained results may impact: *(i)* practitioners, helping them improve their TDD practice through the adoption of proper settings and tools; *(ii)* educators, in better introducing TDD within their courses; and *(iii)* researchers, interested in developing better tool support for developers, or further studying TDD.

CCS CONCEPTS

• **Software and its engineering**;

KEYWORDS

Test-driven Development, Static Analysis Tool, Refactoring

ACM Reference Format:
Simone Romano, Fiorella Zampetti, Maria Teresa Baldassarre, Massimiliano Di Penta, and Giuseppe Scanniello. 2022. Do Static Analysis Tools Affect Software Quality when Using Test-driven Development?. In *ACM / IEEE*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEM '22, September 19–23, 2022, Helsinki, Finland
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9427-7/22/09…$15.00
https://doi.org/10.1145/3544902.3546233

International Symposium on Empirical Software Engineering and Measurement (ESEM) (ESEM '22), September 19–23, 2022, Helsinki, Finland. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3544902.3546233

1 INTRODUCTION

Test-Driven Development (*TDD*) is an agile software development practice in which unit tests are written before production code [7]. More in detail, TDD promotes short cycles, composed of three phases each, to incrementally implement software functionality:

Red Phase. Write a unit test for a small chunk of functionality not yet implemented and watch that unit test fail;

Green Phase. Make that unit test pass as quickly as possible, committing whatever "sin" is necessary to do so, and watch all unit tests pass;

Refactor Phase. Refactor the code, thus remedy any sin previously committed just to make the unit test pass, and watch all unit tests pass.

The Red-Green-Refactor cycle is thus repeated until the functionality is completely implemented. When this happens, the TDD practitioner can tackle new functionality.

Advocates of TDD claim that this agile practice allows improving software quality as well as developers' productivity [19]. These claimed benefits have encouraged some software companies to adopt TDD, while others have been considering its adoption [58]. There is also a wide community around TDD—for example, the "TDD" tag on *Stack Overflow* has 5.2k watchers (in 05/2022). Nevertheless, the results about the claimed benefits, gathered in secondary studies, are not conclusive yet (*e.g.*, [24, 31]).

A key role in TDD is played by refactoring since it allows to constantly improve software quality, while preserving software external behavior [19]. Nevertheless, empirical evidence shows that the Refactor phase is often skipped (*e.g.*, [2, 48, 53]). For example, Romano *et al.* [48] observed that refactoring is not performed as often as TDD requires and the Refactor phase is (wrongly) perceived as less important than the other two (*i.e.*, Red and Green). Skipping the refactor phase is fine only when there is no refactoring opportunity. In any other case, skipping this phase would negatively affect software quality. This might explain inconclusive results on the claimed benefits of TDD on software quality [24, 31, 54].

What triggers actions aimed to improve software quality, including refactoring actions, both inside and outside the TDD process?

Empirical Software Engineering (2023) 28:89
https://doi.org/10.1007/s10664-023-10287-x

Rubbing salt in the wound? A large-scale investigation into the effects of refactoring on security

Emanuele Iannone[1] · Zadia Codabux[2] · Valentina Lenarduzzi[3] · Andrea De Lucia[1] · Fabio Palomba[1]

Accepted: 5 January 2023 / Published online: 24 May 2023
© The Author(s) 2023

Abstract

Software refactoring is a behavior-preserving activity to improve the source code quality without changing its external behavior. Unfortunately, it is often a manual and error-prone task that may induce regressions in the source code. Researchers have provided initial compelling evidence of the relation between refactoring and defects, yet little is known about how much it may impact software security. This paper bridges this knowledge gap by presenting a large-scale empirical investigation into the effects of refactoring on the security profile of applications. We conduct a three-level mining software repository study to establish the impact of 14 refactoring types on (i) security-related metrics, (ii) security technical debt, and (iii) the introduction of known vulnerabilities. The study covers 39 projects and a total amount of 7,708 refactoring commits. The key results show that refactoring has a limited connection to security. However, *Inline Method* and *Extract Interface* statistically contribute to improving some security aspects connected to encapsulating security-critical code components. *Extract Superclass* and *Pull Up Attribute* refactoring are commonly found in commits violating specific security best practices for writing secure code. Finally, *Extract Superclass* and *Extract & Move Method* refactoring tend to occur more often in commits contributing to the introduction of vulnerabilities. We conclude by distilling lessons learned and recommendations for researchers and practitioners.

Keywords Refactoring · Software security · Empirical SE

1 Introduction

In 1999, Fowler defined the term *"software refactoring"* to indicate the activities developers perform to improve the internal structure of source code without changing its external behavior (Martin and Kent 1999). Since then, the research community has investigated refactoring from multiple perspectives (Al Dallal and Abdin 2017; Azeem et al. 2019;

Communicated by: Miryung Kim

✉ Emanuele Iannone
eiannone@unisa.it

Extended author information available on the last page of the article.

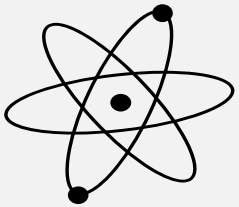
UDESC

Paradigmas de **Pesquisa**

Quali

Estuda crenças e entendimentos

Descobrir o porquê



Quanti

Controlados

Variáveis Planejadas

Testar efeitos

UDESC

Estratégias Empíricas

1) Survey

- Investigação realizada em retrospecto para um objeto (técnica, ferramenta) em uso
- Os dados são obtidos por entrevistas ou questionários
- Usa amostras representativas da população estudada
- Os resultados são analisados para derivar conclusões descritivas e explanatórias
- As conclusões são generalizadas para a população das amostras

Estratégias Empíricas

A Survey on Deep Learning for Software Engineering

YANMING YANG, School of Computer Science and Technology, Zhejiang University, China
XIN XIA, Software Engineering Application Technology Lab, Huawei, China
DAVID LO, School of Information Systems, Singapore Management University, Singapore
JOHN GRUNDY, Faculty of Information Technology, Monash University, Australia

In 2006, Geoffrey Hinton proposed the concept of training "Deep Neural Networks (DNNs)" and an improved model training method to break the bottleneck of neural network development. More recently, the introduction of AlphaGo in 2016 demonstrated the powerful learning ability of deep learning and its enormous potential. Deep learning has been increasingly used to develop state-of-the-art software engineering (SE) research tools due to its ability to boost performance for various SE tasks. There are many factors, e.g., deep learning model selection, internal structure differences, and model optimization techniques, that may have an impact on the performance of DNNs applied in SE. Few works to date focus on summarizing, classifying, and analyzing the application of deep learning techniques in SE. To fill this gap, we performed a survey to analyze the relevant studies published since 2006. We first provide an example to illustrate how deep learning techniques are used in SE. We then conduct a background analysis (BA) of primary studies and present four research questions to describe the trend of DNNs used in SE (BA), summarize and classify different deep learning techniques (RQ1), and analyze the data processing including data collection, data classification, data pre-processing, and data representation (RQ2). In RQ3, we depicted a range of key research topics using DNNs and investigated the relationships between DL-based model adoption and multiple factors (i.e., DL architectures, task types, problem types, and data types). We also summarized commonly used datasets for different SE tasks. In RQ4, we summarized the widely used optimization algorithms and provided important evaluation metrics for different problem types, including regression, classification, recommendation, and generation. Based on our findings, we present a set of current challenges remaining to be investigated and outline a proposed research road map highlighting key opportunities for future work.

CCS Concepts: • **Software and its engineering → Software development techniques;**

Additional Key Words and Phrases: Deep learning, neural network, machine learning, software engineering, survey

This research is supported by ARC Laureate Fellowship (FL190100035) and the National Research Foundation, Singapore under its Industry Alignment Fund – Pre-positioning (IAF-PP) Funding Initiative. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.
Authors' addresses: Y. Yang, School of Computer Science and Technology, Zhejiang University, Hangzhou, China; email: yanmingyang@zju.edu.cn; X. Xia (corresponding author), Software Engineering Application Technology Lab, Huawei, Hangzhou, China; email: xin.xia@acm.org; D. Lo, School of Information Systems, Singapore Management University, Singapore; email: davidlo@smu.edu.sg; J. Grundy, Faculty of Information Technology, Monash University, Melbourne, Australia; email: John.Grundy@monash.edu.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2022 Association for Computing Machinery.
0360-0300/2022/09-ART206 $15.00
https://doi.org/10.1145/3505243

ACM Computing Surveys, Vol. 54, No. 10s, Article 206. Publication date: September 2022.

ELSEVIER

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

The Symposium on Search-Based Software Engineering: Past, Present and Future

Thelma Elita Colanzi[a,*], Wesley K.G. Assunção[b], Silvia R. Vergilio[c], Paulo Roberto Farah[c,d], Giovani Guizzo[e]

[a] DIN - State University of Maringá (UEM), Maringá, Brazil
[b] COTSI, Federal University of Technology - Paraná (UTFPR), Toledo, Brazil
[c] DINF, Federal University of Paraná (UFPR), Curitiba, Brazil
[d] DESO, Santa Catarina State University (UDESC), Joinville, Brazil
[e] CREST Centre - Department of Computer Science, University College London (UCL), London, UK

ARTICLE INFO

Keywords:
Systematic mapping
Search-based software engineering
Bibliometric analysis

ABSTRACT

Context: Search-Based Software Engineering (SBSE) is the research field where Software Engineering (SE) problems are modelled as search problems to be solved by search-based techniques. The Symposium on Search Based Software Engineering (SSBSE) is the premier event on SBSE, which had its 11th edition in 2019.

Objective: In order to better understand the characteristics and evolution of papers published at SSBSE, this work reports results from a mapping study targeting the proceedings of all SSBSE editions. Despite the existing mapping studies on SBSE, our contribution in this work is to provide information to researchers and practitioners willing to enter the SBSE field, being a source of information to strengthen the symposium, guide new studies, and motivate new collaboration among research groups.

Method: A systematic mapping study was conducted with a set of four research questions, in which 134 studies published in all editions of SSBSE, dated from 2009 to 2019, were evaluated. In a fifth question, 32 papers published in the challenge track were summarised.

Results: Throughout the years, 290 authors from 25 countries have contributed to the main track of the symposium, with the collaboration of at least two institutions in 46.3% of the papers. SSBSE papers have got substantial external visibility, as most citations are from different venues. The SE tasks addressed by SSBSE are mostly related to software testing, software debugging, software design, and maintenance. Evolutionary algorithms are present in 75% of the papers, being the most common search technique. The evaluation of the SBSE approaches usually includes industrial systems.

Conclusions: SSBSE has helped increase the popularity of SBSE in the SE research community and has played an important role on making SBSE mature. There are still problems and challenges to be addressed in the SBSE field, which can be tackled by SSBSE authors in further studies.

1. Introduction

Search Based Software Engineering (SBSE) is the research field that formulates Software Engineering (SE) problems as search problems. In this way, heuristic techniques are used to find near-optimal solutions to efficiently solve a large variety of problems associated to different SE tasks. The International Symposium on Search Based Software Engineering (SSBSE)[1] is the premier event on SBSE, with eleven editions so far. Over the past ten years, the symposium has drawn attention of researchers, academics, and practitioners alike, contributing to strengthen the field and to integrate the SBSE community, whilst gathering a large body of studies that are now relevant SBSE references.

Thanks to SSBSE we can say that the SBSE field is a mature and active field of research. In the literature we can find surveys reporting a wide view of SBSE on software bug fixing, project management, planning and cost estimation, software comprehension, refactoring, software slicing, service-oriented software engineering, compiler optimisation, quality assessment, and others [1–3]. Such surveys analyse the most

* Corresponding author.
E-mail addresses: thelma@din.uem.br (T.E. Colanzi), wesleyk@utfpr.edu.br (W.K.G. Assunção), silvia@inf.ufpr.br (S.R. Vergilio), paulo.farah@udesc.br (P.R. Farah), g.guizzo@ucl.ac.uk (G. Guizzo).
[1] https://ssbse.info/.

https://doi.org/10.1016/j.infsof.2020.106372
Received 30 November 2019; Received in revised form 25 June 2020; Accepted 1 July 2020
Available online 2 July 2020
0950-5849/© 2020 Elsevier B.V. All rights reserved.

UDESC

Estratégias Empíricas

2) Estudo de Caso

- São usados para monitorar projetos, atividades e atribuições
- Os dados são coletados para um propósito específico
- Aplica análises estatísticas nos dados
- Normalmente é destinado a rastrear um atributo específico ou a estabelecer relações entre diferentes atributos
- Possui um nível de controle menor do que de um experimento
- Exemplo: Um modelo de predição do número de defeitos em testes. Pode-se usar análise estatística multivariada com análise de regressão linear e PCA (análise de componente principal)

UDESC

Estratégias Empíricas

3) Experimentação

- São realizados em um ambiente de laboratório, com alto nível de controle
- Os objetos de estudo são atribuídos a diferentes tratamentos aleatoriamente
- O objetivo é manipular uma ou mais variáveis e controlar todas as outras com valores fixos
- Os efeitos da manipulação são medidos e analisados estatisticamente
- Exemplo: Comparar dois métodos de inspeção de código. Em seguida, utilizar inferência estatística para mostrar que um método é melhor que o outro com significância estatística.

Paradigmas de **Pesquisa**

- **Survey:**
  - Muito comuns em ciências sociais (ex.: eleições)
  - Não fornece controle da execução ou da medição.
  - Não permite manipular as variáveis

- **Estudo de Caso:**
  - Observa e documenta fatores chaves que afetam o resultado de um projeto em execução

- **Experimento:**
  - É uma investigação formal, rigorosa e controlada
  - O objetivo é distinguir entre valores controlados e investigados de fatores chaves

UDESC

Qualitativo x Quantitativo

Estratégia	**Qualitativo/Quantitativo**
Survey	Ambos
Estudo de caso	Ambos
Experimentos	Quantitativo

UDESC

Survey

- **São realizados quando o uso de uma técnica ou ferramenta já está em uso, ou antes de ser introduzida**
- **Mostra um retrato da situação atual**
- **Pode ser usado para pesquisas de opinião e de mercado**

- **Exemplo:** Estudar como um novo processo de desenvolvimento melhorou as atitudes dos desenvolvedores quanto na qualidade de software. Uma amostra de desenvolvedores é selecionada, um questionário é construído e aplicado e os resultados são organizados e analisados.

Survey - Características

- O objetivo é entender a população de onde a amostra foi obtida
- Surveys tem como foco o desenvolvimento de sugestões genéricas e não casos específicos
- Permite a coleta de muitas variáveis
- É necessário ter o foco em obter o máximo de entendimento com o menor número de variáveis
- Possibilita construir uma variedade de modelos explanatórios para selecionar um que melhor se adequa aos propósitos da pesquisa

UDESC

Survey - Objetivos

- **Os objetivos gerais de uma survey podem ser:**
  - **Descritivos**: permite fazer afirmações sobre a população como determinar qual é a distribuição de atributos e características
  - **Explanatórios**: visam fazer afirmações explicativas sobre a população como explicar o porque uma técnica é preferida do que outra.
  - **Exploratórios**: são usadas como um pré-estudo para uma investigação mais aprofundada para garantir que questões importantes não sejam previstas. Usa um questionário pouco estruturado. Não responde questões mas cria mais possibilidades a serem avaliadas.

UDESC

Survey – Coleta de Dados

- **Questionários**
- **Entrevistas**

UDESC

Survey - Exemplo

1024 IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, VOL. 44, NO. 11, NOVEMBER 2018

Data Scientists in Software Teams: State of the Art and Challenges

Miryung Kim, Thomas Zimmermann, Robert DeLine, and Andrew Begel

Abstract—The demand for analyzing large scale telemetry, machine, and quality data is rapidly increasing in software industry. Data scientists are becoming popular within software teams, e.g., Facebook, LinkedIn and Microsoft are creating a new career path for data scientists. In this paper, we present a large-scale survey with 793 professional data scientists at Microsoft to understand their educational background, problem topics that they work on, tool usages, and activities. We cluster these data scientists based on the time spent for various activities and identify 9 distinct clusters of data scientists, and their corresponding characteristics. We also discuss the challenges that they face and the best practices they share with other data scientists. Our study finds several trends about data scientists in the software engineering context at Microsoft, and should inform managers on how to leverage data science capability effectively within their teams.

Index Terms—Data science, development roles, software engineering, industry

1 Introduction

Software teams increasingly analyze data to inform their engineering and business decisions and to build data usage, topics that they work on, and types of data that they work with.

UDESC

Case Studies

- Investiga uma entidade ou fenômeno único em um espaço de tempo específico
- Adequado para avaliação industrial de métodos e ferramentas de ES
- Evita problemas de escala
- Estudam os casos mais típicos
- São mais fáceis de planejar
- Não conseguem generalizar os resultados tão bem quanto os experimentos

Estudo de Caso

- Pode ser usado para comparar resultados de métodos, abordagens e ferramentas diferentes
- É necessário criar uma base sólida para evitar viés e garantir a validade interna:
  - Comparar resultados de um novo método com um baseline da empresa
  - Comparar projetos com as mesmas características mas com alterações controladas
  - se o método se aplica a componentes individuais do produto, pode ser aplicado aleatoriamente a outros componentes

UDESC

Estudo de Caso

2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)

lysis, Evolution and Reengineering (SANER) | 978-1-7281-9630-5/20/$31.00 ©2021 IEEE | DOI: 10.1109/SANER50967.2021.00042

A Multi-Criteria Strategy for Redesigning Legacy Features as Microservices: An Industrial Case Study

Wesley K. G. Assunção*†, Thelma Elita Colanzi*‡, Luiz Carvalho*, Juliana Alves Pereira*,
Alessandro Garcia*, Maria Julia de Lima§, Carlos Lucena*
*DI – Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro, Brazil.
†PPGComp – Western Paraná State University, Cascavel, Brazil.
‡DIN – State University of Maringá, Maringá, Brazil.
§Tecgraf Institute – Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro, Brazil.

***Abstract*—Microservices are small and autonomous services that communicate through lightweight protocols. Companies have often been adopting microservices to incrementally redesign legacy systems as part of a modernization process. Microservices promote better reuse and customization of existing features while increasing business capabilities, if appropriate design decisions are made. There are some partially-automated approaches supporting the re-design of legacy features into microservices. However, they fail in covering two key aspects: *(i)* provide an architectural design of the features being redesigned, and *(ii)* simultaneously support relevant criteria, *e.g.*, feature modularization and decrease of network communication overhead. Also, these two aspects tend to be poorly discussed along industrial case studies. To fulfill these gaps, we propose a redesign strategy to support the re-engineering of features legacy code as microservices. This strategy covers key possibly-conflicting criteria on microservice-based architectures. We employ search-based optimization to deal with such conflicting criteria. The output of the strategy is a set of redesign candidates of legacy features as microservices. We reflect upon the benefits and drawbacks of the proposed strategy through an industrial case study. In particular, we perform an in-depth analysis of the resulting microservice candidates, and a discussion about their potential for customization and reuse. The reflections/discussions are also supported by observations of developers involved in the process.**

***Index Terms*—microservice architecture, legacy systems, software evolution, search-based software engineering.**

redesign. However, redesigning legacy features into microservices is a quite complex activity for several reasons [9].

The redesign activity should guide: *(i)* the identification of features (and possibly subfeatures) that are likely to be aligned with the business capabilities, *(ii)* the proper modularization of such features into microservices, *(iii)* the simultaneous satisfaction of influencing criteria, such as cohesion and network overhead [10], *(iv)* the generation of alternative designs with different levels of microservice decomposition – alternatives are essential to support maintainers in choosing the one that fits best their needs, and *(v)* the redesign of features that are likely to be reusable and customizable.

There are some partially-automated approaches for supporting the redesign of existing systems into microservices [11, 12, 13, 14] (Section II). However, they are not feature-driven, *i.e.*, developers cannot select an initial set of features to be redesigned as microservices. These approaches also do not support the further decomposition of candidate features into subfeatures, which improve modularization and the alignment with new business capabilities. There are several approaches for feature modularization [15], but they often support only one or two criteria [16, 17], and do not master microservice-specific criteria, *e.g.*, network overhead. Moreover, the evaluation of such approaches does not cover the five guidance needs mentioned above and also neither consider industrial systems.

UDESC

Experimentos

- São realizados quando se deseja ter controle sobre a situação e manipular o comportamento de maneira direta, precisa e sistemática
- Envolvem mais de um tratamento para comparar as saídas. Por exemplo, quem usa um método e quem usa outro, quando e porque
- Podem ser feitos em laboratório simulando o ambiente real ou diretamente no ambiente real
-

Experimentos - Características

- Confirmar teorias e sabedoria popular
- Explorar relacionamentos
- Avaliar a acurácia de modelos
- Validar medições

A força de um experimento é poder investigar em qual situação as afirmações são verdadeiras e podem pode fornecer um contexto no qual certos padrões, métodos e ferramentas são recomendados para uso

Experimentos - Processo

1) Definição
2) Planejamento
3) Operação
4) Análise e interpretação
5) Apresentação

UDESC

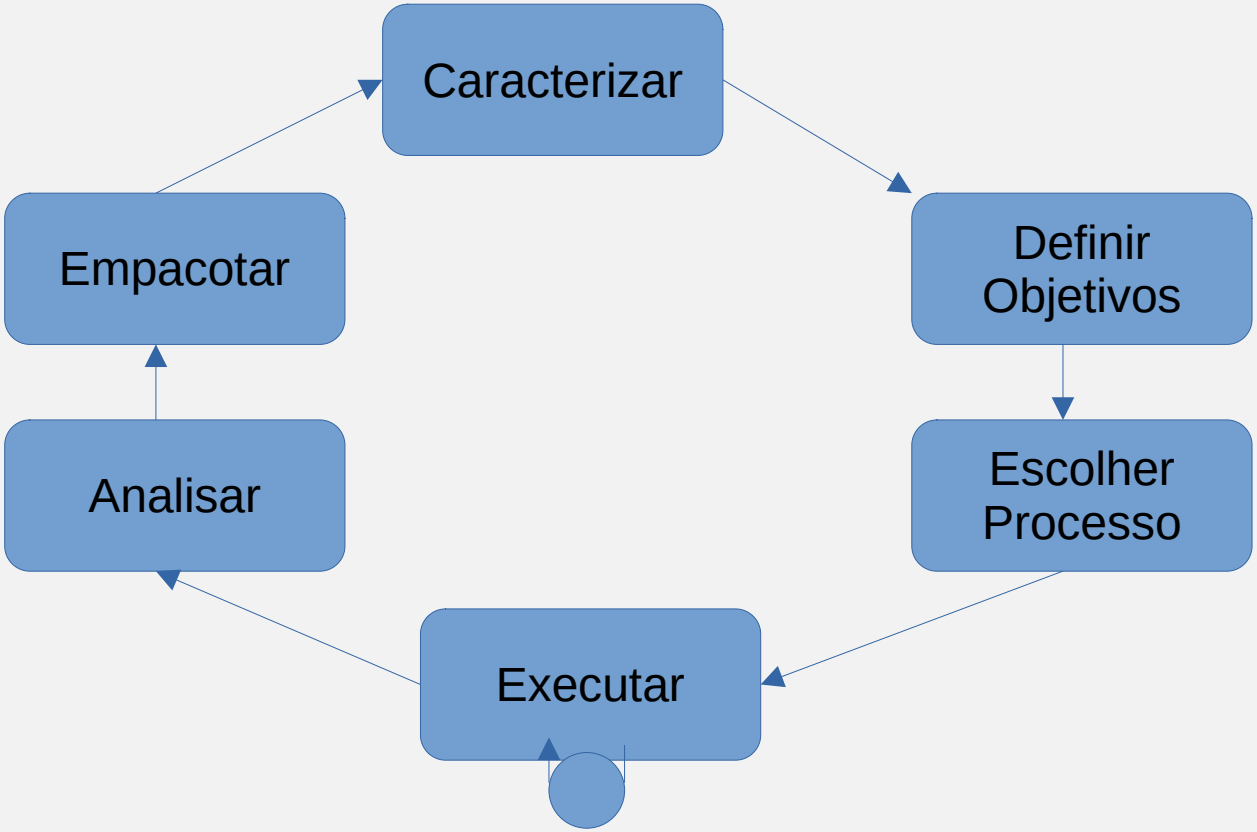
Empirismo em ES

Oportunidade de obter resultados objetivos e estatísticamente significativos para entender, controlar, predizer e melhorar o desenvolvimento de software.

Metodologias:

1) Paradigma de Melhoria de Qualidade (QIP)
2) Fábrica de Experiência
3) Paradigma Goal/Question/Metric (GQM)

Paradigma de Melhoria de Qualidade (QIP)



UDESC

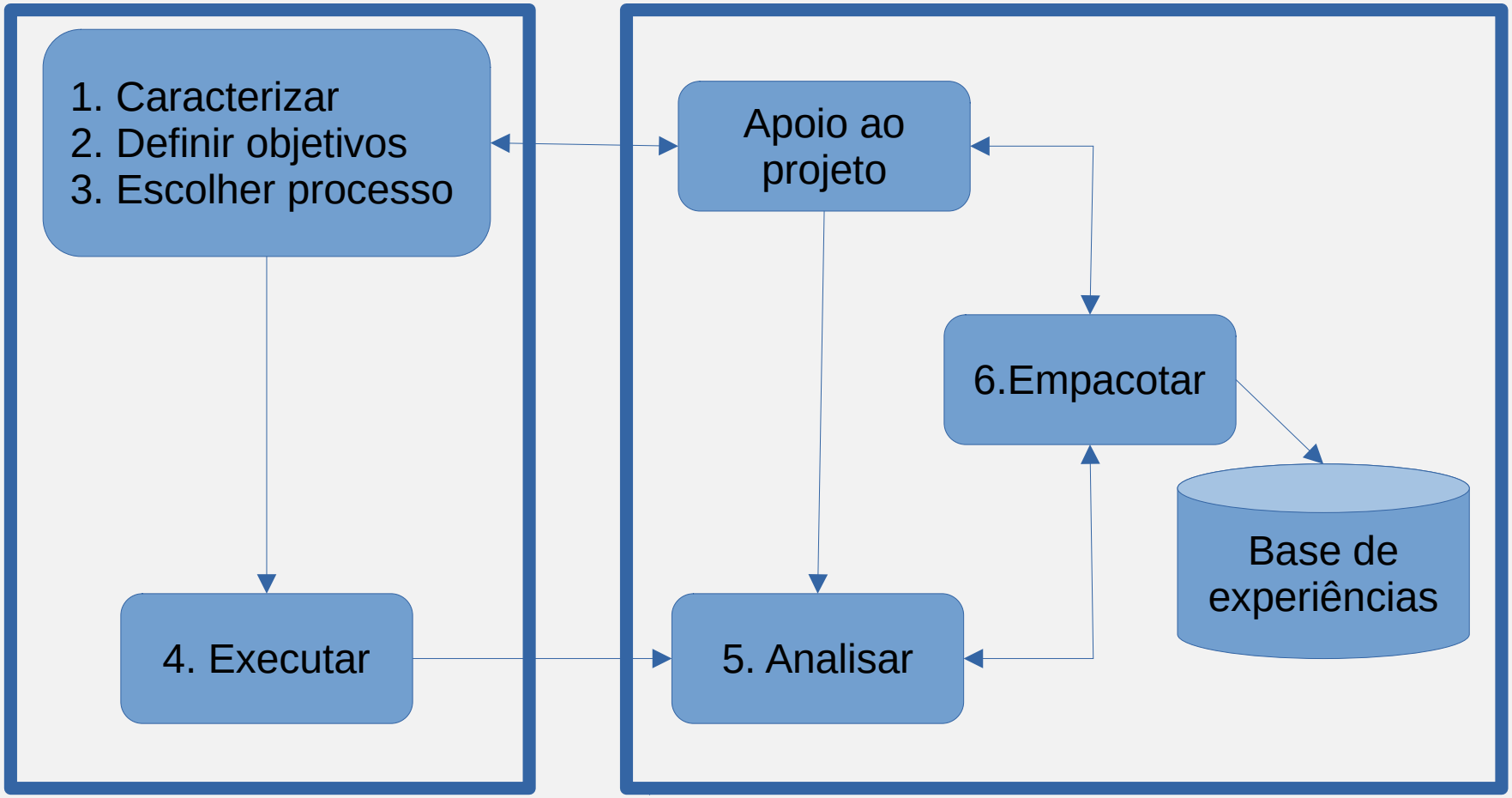
Paradigma de Melhoria de Qualidade (QIP)

1) **Caracterizar**: Entender o ambiente a partir de modelos disponíveis, dados, intuição, etc.
2) **Definir objetivos**: definir objetivos quantificáveis para o sucesso do projeto e para a melhoria da organização
3) **Escolher o processo**: Escolher o processo adequado de acordo com o ambiente e os objetivos escolhidos
4) **Executar**: Desenvolver o produto e dar feedback de acordo com os resultados
5) **Analisar**: No final do projeto, analisar as práticas adotadas, determinar problemas, registrar descobertas a fazer recomendações futuras
6) **Empacotar**: Consolidar a experiência em modelos

Paradigma de Melhoria de Qualidade (QIP)

- Requer aprendizagem contínua
- A experiência deve ser empacotada em modelos de experiência que devem ser efetivamente entendidos e modificados
- Os modelos são armazenados em uma base e podem ser modificados para serem reutilizados nos projetos mais novos

Fábrica de Experiências



Projeto

Fábrica de Experiências

UDESC

Fábrica de Experiências

Empacota experiências criando modelos informais, formais ou esquematizados e mensurados de vários processos, produtos e de outras fontes de conhecimento como pessoas, documentos e automaticamente

UDESC

Fábrica de Experiências

International Journal on Advanced Science Engineering Information Technology

Vol.10 (2020) No. 3
ISSN: 2088-5334

The Development and Evaluation of Experience-Based Factory Model for Software Development Process

Mastura Hanafiah[a,1], Rusli Abdullah[a,2], Masrah Azrifah Azmi Murad[a,3], Jamilah Din[a,4]

[a] *Faculty of Computer Science and Information Technology, Universiti Putra Malaysia, 43400 Malaysia*
E-mail: [1]mastura.hanafiah@daimler.com; [2]rusli@upm.edu.my; [3]masrah@upm.edu.my; [4]jamilahd@upm.edu.my

***Abstract*— Knowledge, and experiences in software development have been accumulated over time throughout the project lifecycle. Previous studies have shown that the management of knowledge and experiences in software development has always been an issue. Therefore, the knowledge transfer and information flow are inefficient, misinterpretation, and inconsistencies always occur between individuals or teams, and the organization fails to learn from past projects. It is understood that efficient knowledge and experience management for software development organizations is crucial for the purpose of sharing and future reuse. This paper discusses the prototype development for a proposed model, which is based on the experience factory approach, to manage knowledge and experiences for the software development process. Discussions include the system functionalities and design, infrastructure requirements, and implementation approach. The efficiency and effectiveness of the prototype are evaluated as survey research based on Jennex & Olfman knowledge management success model. Rasch analysis is used for data reliability and validity. Results show positive feedback on the model's efficiency and effectiveness. Additionally, as agreed by most respondents, the top three of the model contributions are: to encourage learning organization, to prevent knowledge loss and to aid in decision making.**

***Keywords*—experience factory; knowledge management; software development process; prototype evaluation.**

I. Introduction

In software development (SD), there are many stages of events and activities take place typically; some of them are iterative in nature. Throughout the development lifecycle, a lot of methods, techniques, and tools are used. The knowledge and experiences gained during development have become important assets in software organizations. In the previous work, via a systematic literature review [1], it has been identified that there are issues in knowledge management for SD, and it is even more challenging for distributed teams. The main challenges identified are inefficient knowledge transfer and information flow [2], [3], misinterpretation, and inconsistencies [4], [5], and additionally, organizations fail to learn [6]. Due to the importance of knowledge retention and reuse, it is therefore essential to facilitate knowledge management for software process improvements in software organization [2], [7].

Knowledge management (KM) for software development has emerged since the late 1990s, and enormous studies have emerged since then. The interactions between tacit and explicit knowledge allow the conversion of data and

In general, organizational KM has continuously strived for process improvement. Experience Factory (EF) is a software process improvement framework based on reuse of products, processes, and experiences originating from the system lifecycle [9]. EF focuses on two distinctive organization (Fig. 1):

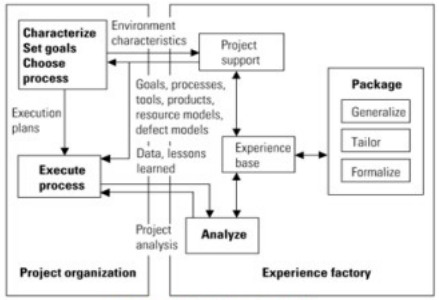


Fig. 1 The Experience Factory [9]

UDESC

Goal/Question/Metric (GQM)

1) Especificar os objetivos próprios e do projeto
2) Rastrear esses objetivos para os dados que definem os objetivos
3) Fornecer uma estrutura para interpretar os dados em relação aos objetivos estabelecidos

Goal/Question/Metric (GQM)

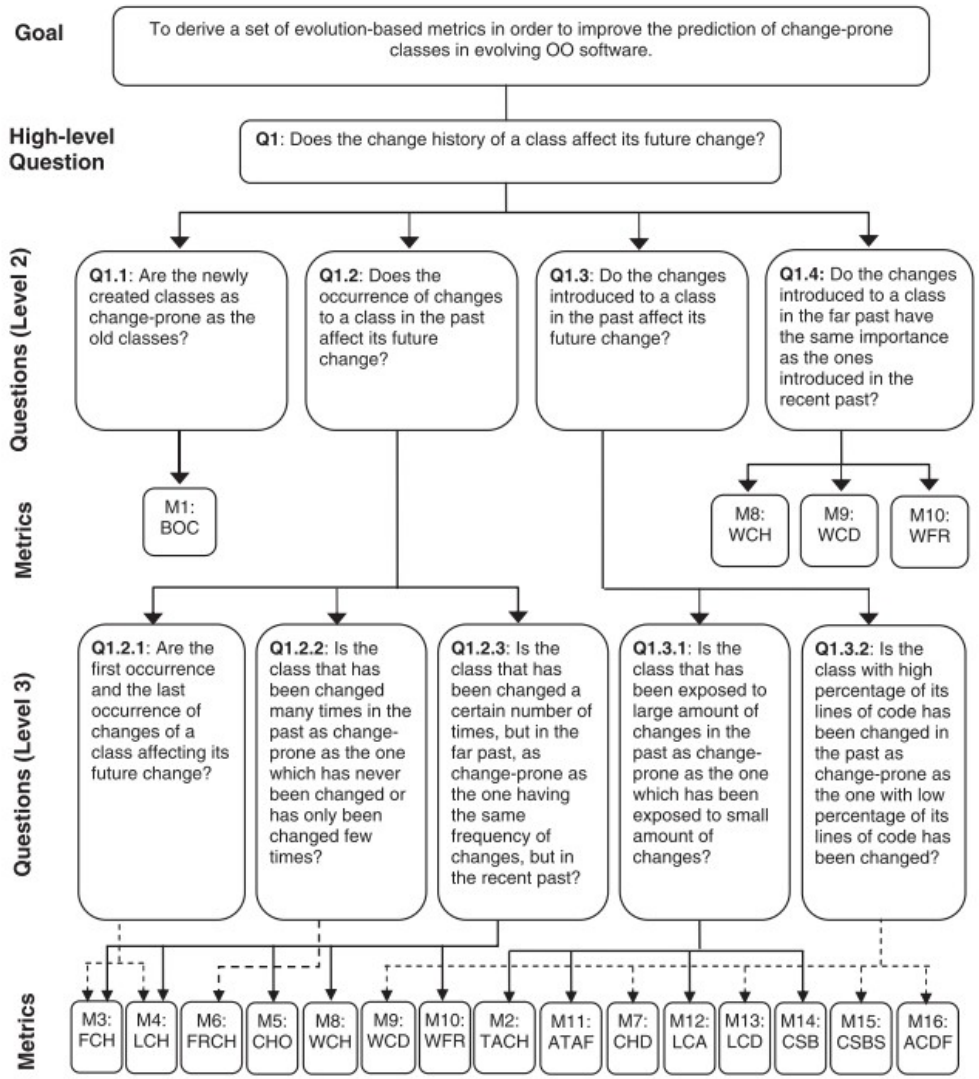


Figure 1. Deriving evolution-based metrics using GQM approach.

UDESC

Medições

“Medição é um mapeamento do mundo empírico para o mundo formal e relacional”

“Uma medida é o número ou símbolo atribuído a uma entidade desse mapeamento para caracterizar um atributo”

(Fenton, 1996)

UDESC

Medições

- “Você não pode controlar o que não pode medir” (DeMarco, 1982)
- A medição é um item central nos estudo empíricos
- O mapeamento de um atributo para um valor mensurado pode ser feito de muitas maneiras (metros, centímetros, etc)
- Cada diferente mapeamento é uma **escala**
- Os valores podem ser transformados de uma escala para outra

Escalas

- **Nominal**: mapeia o atributo da entidade em um nome ou símbolo para classificá-lo

  Exemplos: classificação, rotulação.
- **Ordinal**: ranqueia as entidades por um critério de ordenação

  Exemplos: notas, complexidade de software
- **Intervalar**: trata da diferença entre duas medições

  Exemplo: temperatura
- **Razão**: usa a razão entre dois valores

  Exemplo: velocidade, taxa de transmissão

Classes de Medição

- **Objetiva**: medições que não possuem nenhum tipo de julgamento do valor medido e só depende do objeto que está sendo medido

  Exemplo: Linhas de código (LOC)

- **Subjetiva**: nessas medições, a pessoa que está medindo julga o valor medido. A medição depende do objeto e depende do ponto de vista do medidor

  Exemplos: habilidades pessoais, usabilidade

Medições Diretas e Indiretas

- **Direta**: é capaz de medir um atributo diretamente e não envolve outros atributos

  Exemplo: Linhas de código (LOC), número de defeitos

- **Indireta**: envolve a medição de um atributo é derivada de outros atributos

  Exemplos: densidade de defeitos (#defeitos/LOC), produtividade do programador (LOC/esforço)

Medições em ES

- **<u>Objetos</u>**:
  - **Processo**: descreve as atividades necessárias para produzir software
  - **Produto**: são os artefatos, entregáveis ou documentos resultantes de uma atividade do processo
  - **Recursos**: objetos como pessoas, hardware ou software necessários para atividades do processo

Medições em ES

- **<u>Atributos</u>**:
  - **Internos**: pode ser medido puramente em termos do objeto
  - **Produto**: só podem ser medidos em como o objeto se relaciona com outros objetos

Medições em ES

Classe	Exemplo	Atributo	Exemplo de medição
Produto	Código	Interno	Tamanho
		Externo	Confiabilidade
Processo	Teste	Interno	Esforço
		Externo	Custo
Recurso	Pessoal	Interno	Idade
		Externo	Produtividade

UDESC